

Resumen Objetos 2

1

REFACTORING

↓
transformación de código que preserva el comportamiento pero mejora el diseño.

Mecánicas: conjuntos atómicos de pasos para realizar la transformación

- usos
- eliminar duplicaciones
 - simplificar lógicas complejas
 - clarificar código

LA MEJOR MANERA DE
OPTIMIZAR UN PROGRAMA,
ES PRIMERO ESCRIBIR UN
PROGRAMA BIEN ESTRUCTURADO
Y LUEGO OPTIMIZARLO

NO AÑADE FUNCIONALIDADES

CODE SMELL / BAD SMELL

- ✓ indicadores de posibles problemas en el diseño del código fuente
- ✓ destacan áreas en las que el código podría estar violando principios de diseño fundamentales
- ✓ no son errores, pero futuros desarrollos pueden ser más difíciles
— demandar más horas → costo

VELOCITY

- ✓ cantidad de tareas (funcionalidad) / período de tiempo

DEUDA TÉCNICA

- ✓ costo de rehacer una tarea que tiempo atrás se resuelta con una solución rápida y desprecia

PASES DE UN REFACTORING

- ✓ verificar precondiciones
- ✓ transformaciones
- ✓ compilar & Testear!

IMPORTANCIA DEL REFACTORING

- ✓ generar en la comprensión del código
- ✓ reducir el costo de mantenimiento debido a los cambios inevitables que sufrirá el sistema
- ✓ facilitar la detección de bugs
- ✓ clave: poder agregar funcionalidad más rápido después de refactorizar
- ✓ permite recrear **CLEAN** code

Cohesive
Loosely coupled
Encapsulated
Assertive
Non-redundant

además, LÉGIBLE.

∴ Se toma un código que "huele mal" producto de mal diseño (código duplicado, ilegible, complicado). y se lo trabaja para obtener un buen diseño

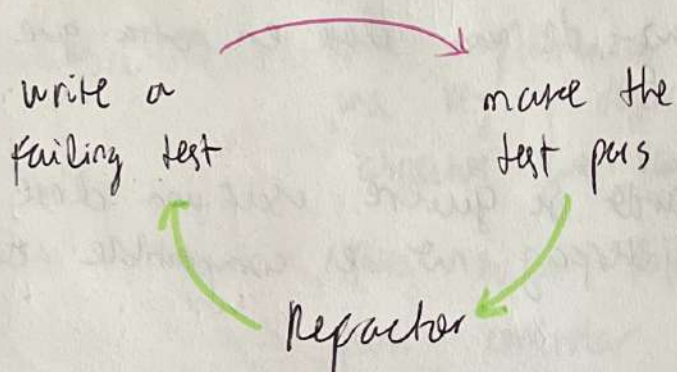
ALGUNOS BAD SMELLS

- duplicated code
- large class
- long method
- data class
- feature envy
- long parameter list
- switch statements

CUANDO APLICAR REFACTORING

2

- en el contexto de TDD
- cuando se ~~descubre~~ descubre código con mal olor, aprovechando la oportunidad
- cuando no puedo entender el código
- cuando encuentro una mejor manera de codificar algo



THE 2 MATS

ADDING FUNCTION

Se exploran ideas,
se corrigen bugs

REFACTORING

Solo puedo refactorizar
Con tests en verde

puedo cambiar de sombrero frecuente //
pero solo puedo usar 1 sombrero a la vez

AUTOMATIZACIÓN DEL REFACTORING

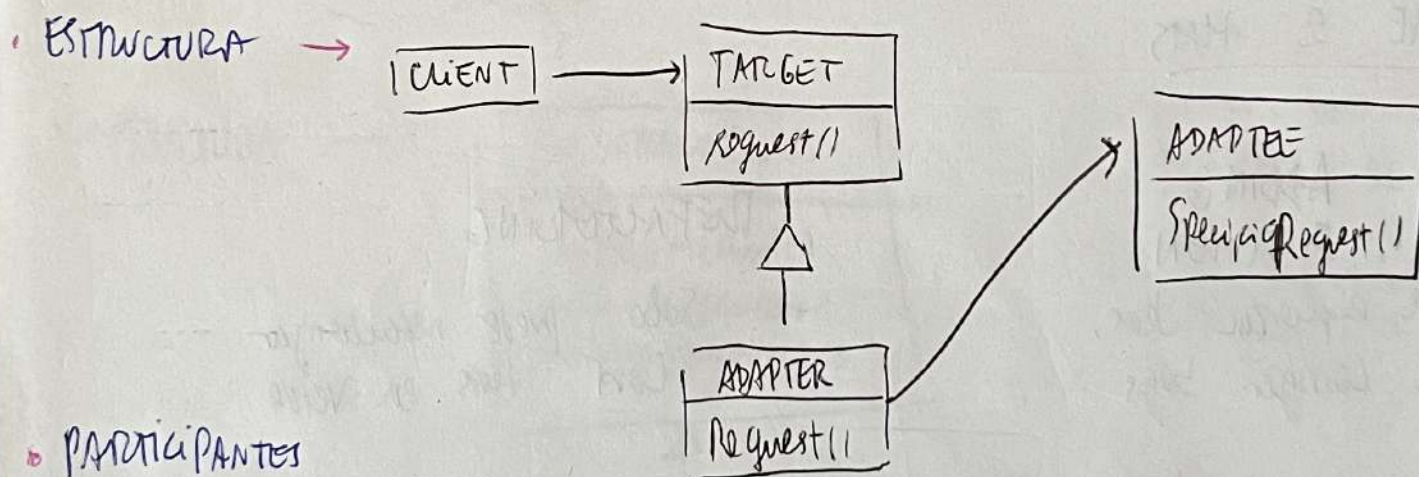
- refactorizar a mano es demasiado costoso: lleva tiempo y puede introducir errores
- Herramientas de refactoring
 - potentes para refactorings chicos
 - restrictivos p/ preservar complejidad del programa
- refactoring automático nace con SMALLTALK
 - ↓ ser pueden demasiado conservativas
 - interactivos → de manera que el chequeo de precondiciones no debe ser extenso

PATRONES DE DISEÑO

- ✓ un patron es un par problema - solución
- ✓ los patrones tratan con problemas recurrentes, tienen soluciones (probados) a estos problemas.
- ✓ la solución es suficiente// genérica para poder aplicarse de $\neq$ maneras.

ADAPTER (WRAPPER)

- INTENCIÓN → "convertir" la interfaz de una clase en otra que el cliente espera
- APLICABILIDAD → usar Adapter cuando se quiere usar una clase existente y su interfaz no es compatible con lo que precisa



• PARTICIPANTES

Target → define la interfaz específica que usa el cliente

Client → colabora con objetos que satisfacen la interfaz de target

Adaptee → define una interfaz que precisa ser adaptada

Adapter → adapta la interfaz del Adaptee a la interfaz de Target

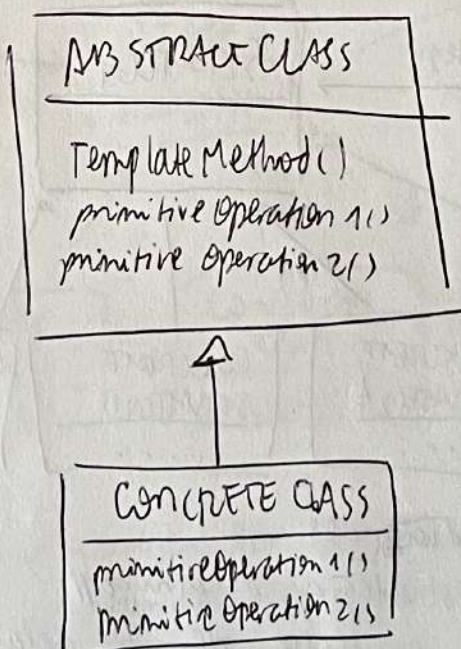
TEMPLATE METHOD

3

- **INTENCIÓN** → definir el esqueleto de un algoritmo en un método, dejando algunos pasos a las subclasses
→ permite que las subclasses redefinan ciertos pasos de un algoritmo sin cambiar la estructura del algoritmo

- **APLICABILIDAD** → para implementar las partes invariantes de un algoritmo una vez y dejar que las subclasses implementen los aspectos que varían
→ para evitar duplicación de código en subclasses
→ para controlar las extensiones que pueden hacer las subclasses

• **ESTRUCTURA** →



• TÉCNICA FUNDAMENTAL DE REUSO DE CÓDIGO

• TIENE INVERSIÓN DE CONTROL

• **PARTICIPANTES**

ABSTRACT CLASS

→ implementa un método que contiene el esqueleto de un algoritmo (Template). Este método llama a operaciones primitivas así como operaciones definidas en Abstract Class

CONCRETE CLASS

→ declara operaciones primitivas abstractas que las subclasses concretas deben definir para implementar los pasos del algoritmo
→ implementa operaciones primitivas que llevan a cabo los pasos específicos del algoritmo

STRATEGY

• INTENCIÓN

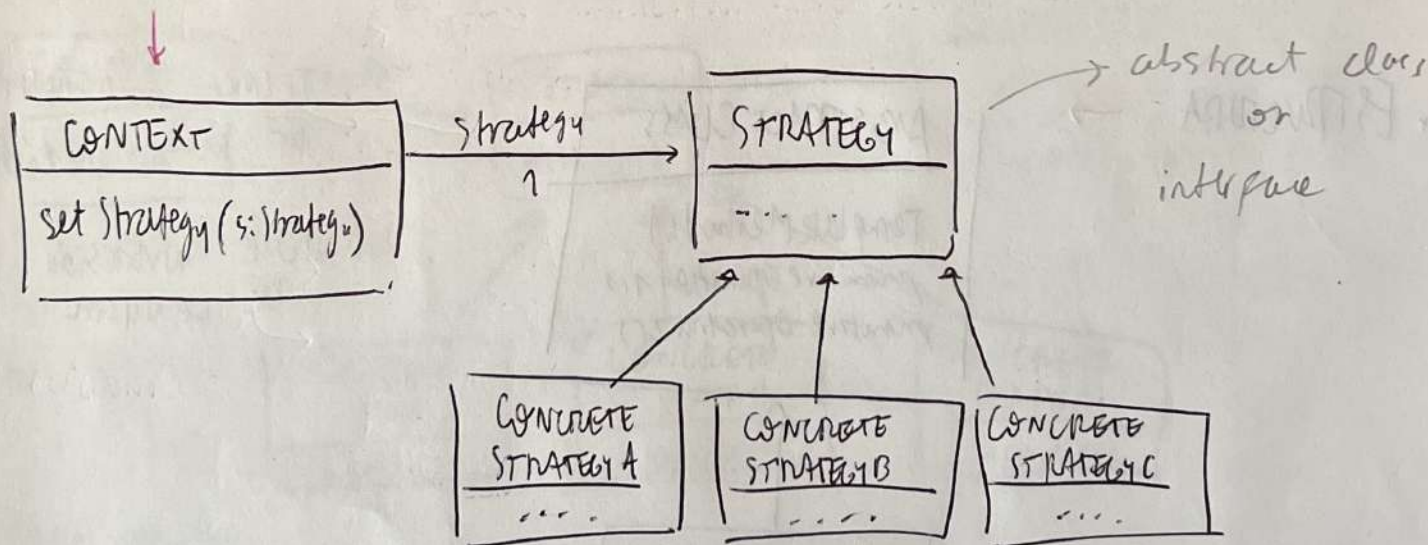
- definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables
- permite que el algoritmo varíe independiente// de los clientes
- permite cambiar (dinámica//) el algoritmo que se utiliza
- brinda flexibilidad para agregar nuevos algoritmos que lleven a cabo una función determinada

• APLICABILIDAD

- cuando existen muchos algoritmos p/ llevar a cabo una tarea
- cuando no es deseable codificar los casos en una clase y seleccionar cuál utilizar con condicionales
- cada algoritmo utiliza info. propia → colocar esto en los clientes lleva a tener clases complejas y difíciles de mantener
- es necesario cambiar el algoritmo en tiempo de ejecución

PARTE CON ADAPTER Y TEMPLATE METHOD

• ESTRUCTURA



VENTAJAS

- elimina condicionales
- cambio de estrategia dinámica//
- desacopla al contexto de los detalles de implementación de las estrategias

DESVENTAJAS

→ Cliente debe conocer las $\neq$ estrategias para poder elegir

• IMPLEMENTACIÓN

- contexto debe tener método para cambiar la estrategia
- parámetro entre el contexto y la estrategia

STATE

• INTENCIÓN

modificar comportamiento de un objeto cuando su estado interno se modifica

externall parecería que la clase del objeto cambió

• APLICABILIDAD

cuando el comportamiento de un objeto depende del estado en el que se encuentre

cuando los métodos tienen condicionales complejos que dependen del estado.

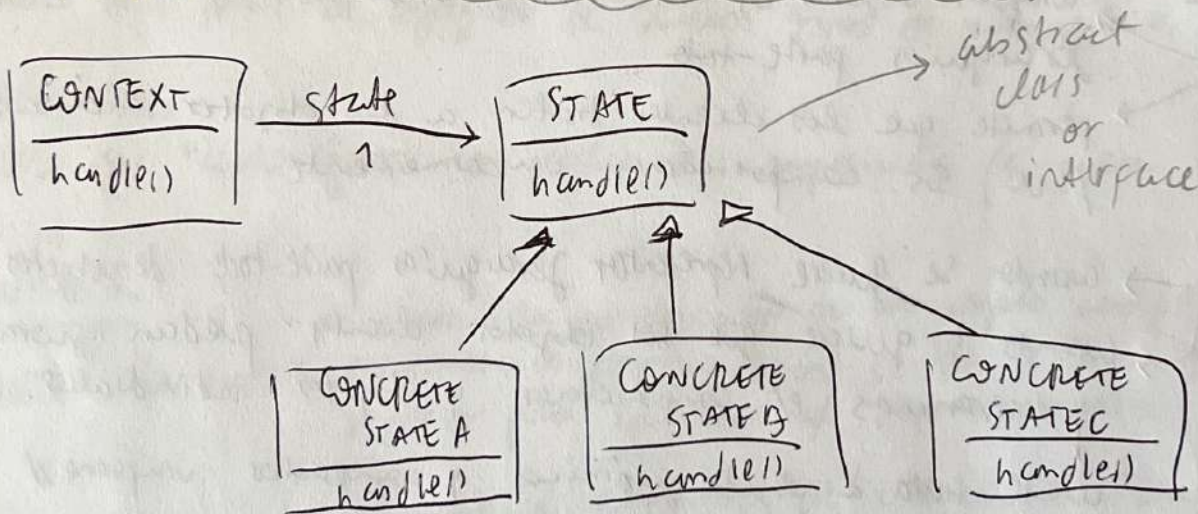
se reemplaza el condicional por clases (uso del polimorfismo)

cada clase concreta representa un estado del objeto

msgs que dependen del estado son delegados a las clases concretas de la jerarquía

los estados son internos al contexto

• ESTRUCTURA



PARTICIPANTES

CONTEXT → interfaz que conocen los clientes
↳ mantiene instancia de algún concrete state

STATE → interfaz para encapsular comportamientos de los estados

CONCRETE STATE → cada subclase implementa el comportamiento específico

VENTAJAS → transiciones explícitas e/ estados
↳ localiza comportamientos de cada estado
↳ estados pueden compartirse (si no tienen variables de instancia)

DESVENTAJAS → alto acoplamiento e/ **CONCRETE STATES** xq la transición es e/ ellas (dependen unas de otras)

STATE VS. STRATEGY

STATE

el estado es privado del objeto

cada state puede definir muchos msgs

los estados concretos se conocen e/ sí. saben a cual estado se debe pasar en respuesta a un msg

STRATEGY

el strategy suele setearse por el cliente, que debe conocer las estrategias posibles

un strategy suele tener un único msg público

los strategies concretos no

COMPOSITE

• INTENCIÓN

componer objetos en estructuras de árbol para representar jerarquías parte-todo

permite que los clientes traten a los objetos atómicos y a sus composiciones uniformemente

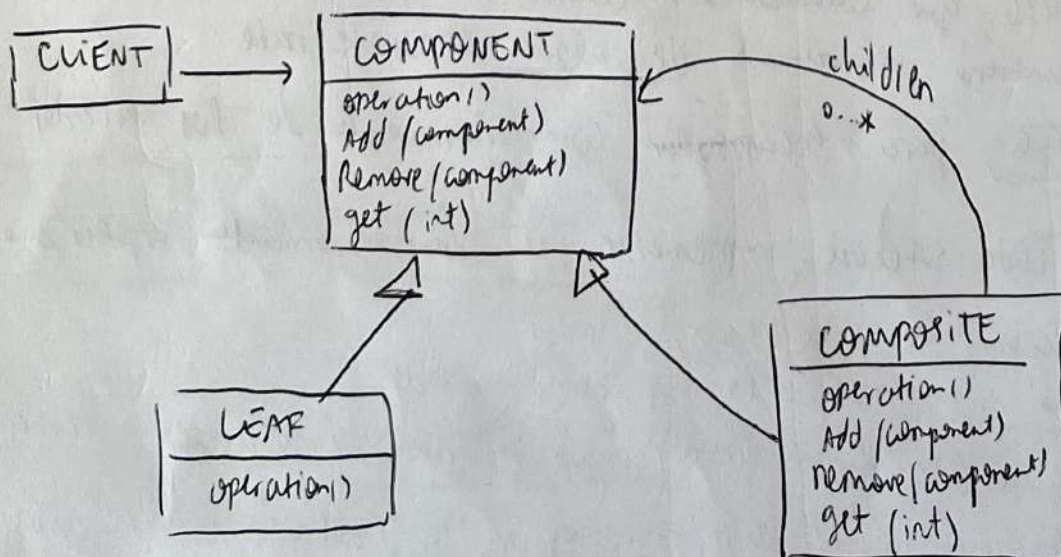
• APLICABILIDAD

→ cuando se quiere representar jerarquías parte-todo de objetos

→ cuando se quiere que los objetos "clientes" puedan ignorar las diferencias e/ composiciones y objetos individuales.

cliente trata a objetos atómicos y compuestos uniforme //

• ESTRUCTURA



• PARTICIPANTES

Component

- declara interfaz para los objetos de la composición
- implementa comportamiento default para la interfaz común a todas las clases
- declara interfaz para definir y acceder "partes de la composición"

Leaf

- los hojas no tienen sub-árboles
- define comportamiento de objetos primitivos en la composición

Composite

- define comportamiento para componentes complejos
- implementa operaciones para manejar sub-árboles

VENTAJAS

- objetos primitivos pueden componerse en objetos complejos, los que a su vez pueden componerse recursivamente
- simplifica los objetos cliente → tratan objetos simple y compuesto de la misma manera
- hace más fácil el agregado de nuevos tipos de componente

DESVENTAJAS

- debe ser "customizado" con reglas de composición (si fuera necesario)

BUILDER

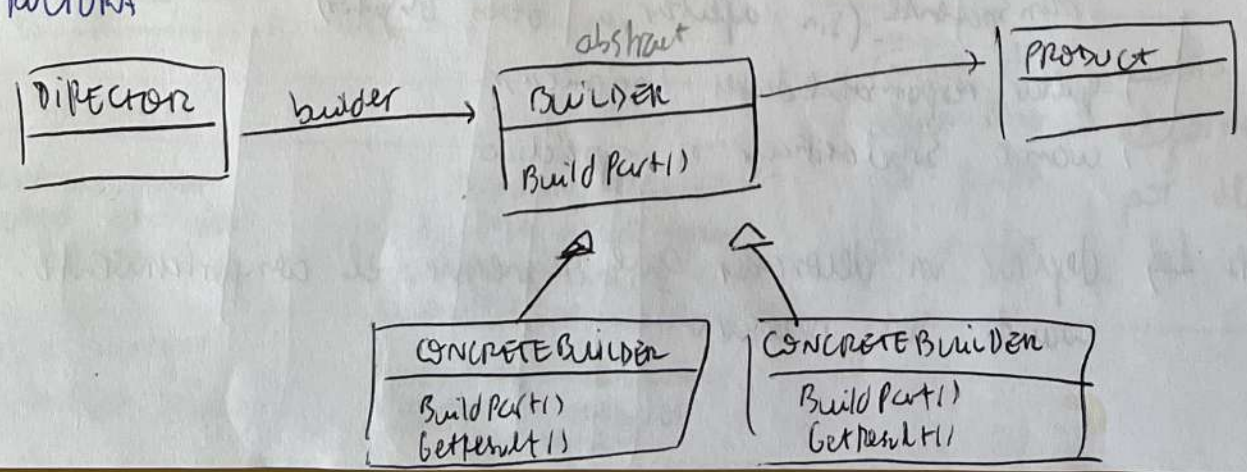
• INTENCIÓN

- separa la construcción de un objeto complejo de su representación (implementación) de tal manera que el mismo proceso puede construir $\neq$ representaciones

• APLICABILIDAD

- cuando el algoritmo de creación de un objeto complejo debería ser independiente de las partes que lo hacen al objeto y cómo se ensamblan

• ESTRUCTURA



• PARTICIPANTES

Builder → especifica una interfaz abstracta para crear partes de un producto

Concrete Builder → construye y ensambla partes del producto

Director → conoce los pasos para construir el objeto

↳ utiliza el builder para construir las partes que van ensamblando

Product → es el objeto completo a ser construido

VENTAJAS

- abstrae la construcción compleja de un objeto complejo
- permite variar lo que se construye Director $\leftrightarrow$ Builder
- da control sobre los pasos de construcción

DESVENTAJAS

- requiere diseñar e implementar varios roles
- cada tipo de producto requiere un concrete builder
- Builder suelen cambiar ($>$ complejidad)

CONSIDERACIONES

- el director sólo sabe hacer una cosa
- $\neq$ directores pueden usar los mismos builders
- Nueva recta → Nuevo director
- Nuevos productos → Nuevo builder

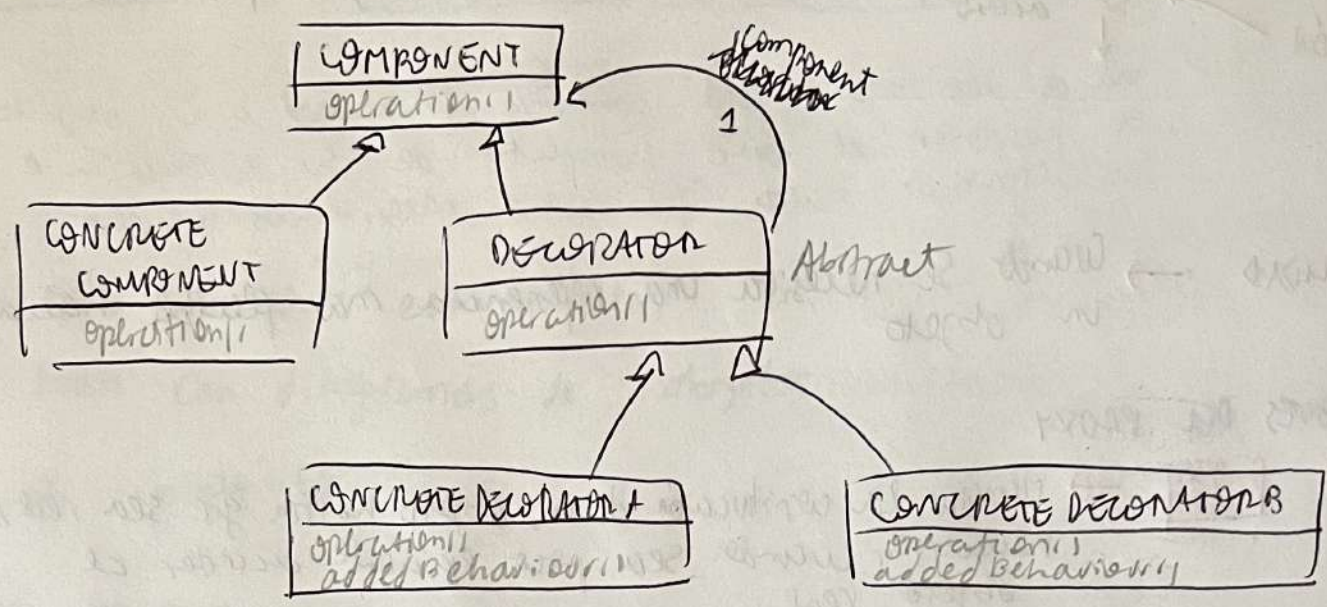
DECORATOR (WRAPPER)

• **INTENCIÓN** → agregar comportamiento a un objeto dinámicamente y en forma transparente

- **APLICABILIDAD** → para agregar responsabilidades a objetos individuales y en forma transparente (sin afectar a otros objetos)
 - ↳ quitar responsabilidades dinámicamente
 - ↳ cuando subclasificar es impráctico

Solución → definir un decorador que agregue el comportamiento cuando sea necesario

ESTRUCTURA



- VENTAJAS → mayor flexibilidad que la herencia
→ permite agregar funcionalidad incremental
- DESVENTAJAS → mayor cantidad de objetos → complejo para depurar
- IMPLEMENTACIÓN
 - ✓ misma interfaz entre componente y decorador
 - ✓ no hay necesidad de Decorador abstracta si se tiene 1 solo decorador
 - ✓ Decorator puede verse como un cambio de "piel"
 - ↳ cambia el comportamiento externo, NO la estructura interna

DECORATOR	ADAPTER
preserva la interfaz del objeto p/ el cliente	convierte la interfaz del objeto p/ el cliente
crean y suelen anidarse	no se anidan

DECORATOR	STRATEGY
propósito: permitir que un objeto cambie su funcionalidad dinámica/ (agregando o cambiando el algoritmo)	
Cambia el algoritmo por fuera del objeto	Cambia el algoritmo por dentro del objeto

DECORATOR	COMPOSITE
mantiene una estructura con solo un siguiente	mantiene estructura de tipo árbol → un composite usa // se compone de varias partes
propósito: agregar funcionalidad dinámica //	propósito: componer objetos y tratarlos uniformemente

Proxy (WRAPPER)

INTENCIÓN

proporcionar un intermediario de un objeto para controlar su acceso

una de las razones para controlar el acceso a un objeto es posponer el costo completo de su creación e inicialización hasta que realmente necesitamos usarlo

APLICABILIDAD

→ Cuando se necesita una referencia más flexible hacia un objeto

APLICACIONES DEL PROXY

Virtual proxy

→ demorar la construcción de un objeto hasta que sea realmente necesario, cuando sea poco eficiente acceder al objeto real

Protection proxy

→ restringir el acceso a un objeto por seguridad

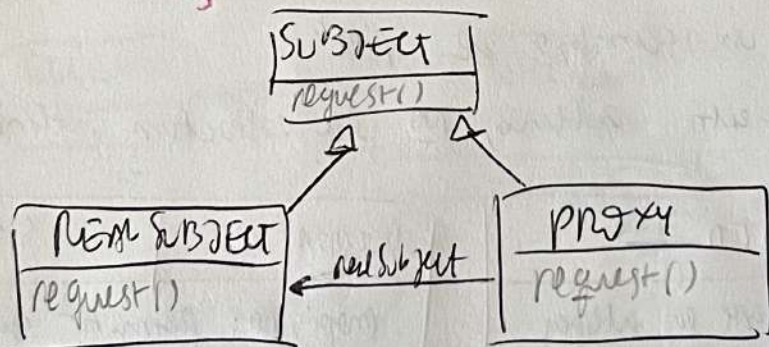
Remote proxy

→ representar un objeto remoto en el espacio de memoria local

→ forma de implementar objetos distribuidos

→ se ocupa de la comunicación con el objeto remoto, de serializar/deserializar los msgs y resultados

ESTRUCTURA



VENTAJAS

→ introduce un nivel de indirección que permite controlar el acceso al objeto real, lo que ofrece flexibilidad en su uso

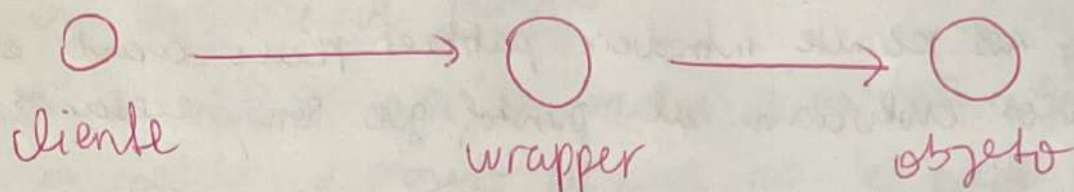
DESVENTAJAS

→ complejidad adicional

PROXY	ADAPTER	PROXY	DECORATOR
proporciona la misma interfaz que el sujeto	proporciona una interfaz $\neq$ al objeto que adapta	controla el acceso a un objeto	agrega una o más responsabilidades a un objeto
		$\neq$ monóitos	

WRAPPERS

→ Adapter, Decorator, Proxy



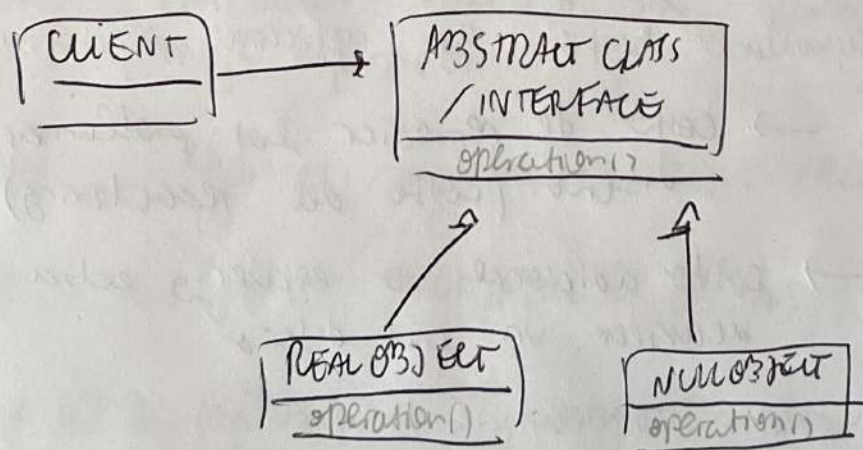
- ✓ todos patrones estructurales
- ✓ todos con diagramas de objetos similares
- ✓ ≠ propósito

NULL OBJECT

INTENCIÓN → proporciona un sustituto para otro objeto que comporte la misma interfaz pero no hace nada

APLICABILIDAD → se usa cuando un objeto requiere un colaborador
↳ Null Object encapsula las implementaciones de "no hacer nada" y oculta esos detalles de sus colaboradores

ESTRUCTURA



IMPLEMENTACIÓN

- ✓ elimina todos los condicionales que verifican si la referencia a un objeto es NULL
 - ✓ hace explícito elementos del dominio que hacen NADA
- puede obviarse la interfaz/clase abstracta y hacer que el Null Object sea subclase de RealObject.

REFACTORING TO PATTERNS

↳ el refactoring nos permite introducir patrones recién cuando el SW que construimos evoluciona al punto que son necesarios

- Form Template Method
- Extract Adapter
- Replace Implicit Tree with Composite
- Replace Conditional Logic with Strategy
- Replace State-altering Conditionals with State
- Move Embellishment to Decorator
- Introduce Null Object

DEUDA TÉCNICA (TD: Technical Debt)

- está bien tomar prestado o endeudarse cuando estratégicamente conviene entregar código rápido para generar feedback
- cada minuto dedicado a código que acumula deuda cuenta como interés → si sigue acumulando hasta que aplicamos refactoring

Capital de la deuda → costo de remediar los problemas de diseño (costo del refactoring)

interés de la deuda → costo adicional o esfuerzo extra por acumular un mal diseño

TEST DE UNIDAD

- ✓ testeo de la mínima unidad de ejecución / funcionalidad
- ✓ en OOP → la unidad mínima es un método
- ✓ **objetivo** → aislar cada parte de un programa y mostrar que funciona correctamente
- ✓ cada test confirma que un método produce el output esperado ante un input conocido
- objetivo:** minimizar los casos a testear y maximizar la cobertura

TEST DOUBLE

8

- ↳ es un lenguaje de patrones
- ↳ cuando es necesario realizar pruebas de un SUT que depende de un módulo u objeto (el cual no se puede utilizar en el ambiente de la prueba)
- ↳ las pruebas pueden ejercitar
 - Configuraciones válidas del sistema
 - "salidas indirectas" del sistema
 - lógica del sistema
 - protocolo
- ↳ Crear un objeto que es una maqueta (polimórfica) del objeto o módulo requerido
- ↳ utilizar la maqueta según se necesite

PATRONES (cascarón vacío → simulación)

- Test Stub → cascarón vacío. Sirve para que el SUT envíe los msgs esperados
- Test Spy → Stub + registro de msgs recibidos
- Mock Object → Spy + verificación de msgs recibidos
- Fake Object → imitación. Se comporta como el módulo real (protocolos, tiempos de respuesta, etc).

Implementar clases según sea necesario

Implementación

- Stub → simple y barato
- Fake Object → demanda análisis, threading, requiere mantenimiento

FRAMEWORKS

- ↳ un framework es una aplicación "semi-completa", "reusable", que puede ser especializada para producir aplicaciones a medida.
- ↳ un conjunto de clases concretas y abstractas, relacionados para proveer una arquitectura reusable que implementa una familia de aplicaciones (relacionados)

CASA BLANCA → Por herencia

- la instanciación hereda y completa el loop de control
- es posible que requiera agregar métodos a clases del Framework
- demanda conocimiento del código del framework
- ↳ las instancias modifican o extienden el código fuente (loop de control + hook clases)

CASA NEGRA → por composición

- utilizan composición (no herencia ni conocimiento del código)
- ↳ las instancias se basan en configuraciones.
- pueden usarse instanciando objetos y componiéndolos (sin entender cómo esas cosas funcionan o quién le manda más a quién)

HotSpot (comportamiento variable)

- ↳ estructura en el código que permite modificar el comportamiento del framework, para instanciar y para extender
- ↳ elementos del diseño pensados para adaptarse

PROTON SPOT (comportamiento invariable)

9

- ↳ aspecto del framework que afecta a todos las instancias y que no se puede modificar (marca indeleble)
- ↳ partes del diseño que no cambian

INVERSIÓN DE CONTROL

- ↳ este principio de diseño permite que el flujo de control del programa sea invertido
- ↳ un framework toma el control y llama a los componentes según sea necesario
- ↳ permite al framework determinar qué set de métodos específicos de la aplicación invocar en respuesta a eventos externos

TIPOS DE FRAMEWORKS

Aplicación `denbor`, `webapps`, `tcp servers`

Manejo Datos `ondb`, `pipelines`, `NRDB`

Sistemas Distribuidos `messages`, `eventos`, `rpc`

Testing `unit`, `webpages`

Singlethread TCP Server (WhiteBox)

- ↳ un servidor como extensión de una clase `loop` + sesión (singular)

Tcp.server.reply (BlackBox)

- ↳ `loop(sesión (msgHandler))`. Sesión singular o múltiple. Diferentes `msgHandler`

▪ `java.util.logging` Jerarquía de labels + composición de `pattern`, `formater` y `salidas`

JAVA.UTIL.LOGGING

- agregamos código de logging a nuestra aplicación para entender lo que pasa con ella
 - ↳ reportes de eventos, errores y excepciones
 - ↳ inicio/fin de operaciones complejas
- útiles para desarrolladores, administradores y usuarios
mucho mejor que System.out.print

la aplicación → configura el **primeron**
↳ manda msg al objeto **logger**

el **primeron** se encarga de

- cómo se crean, organizan y recuperan los objetos
- cómo se configuran
- cómo se activan/desactivan
- a qué prestan atención y a qué no
- cómo se formatean los logs
- a dónde se envían los logs

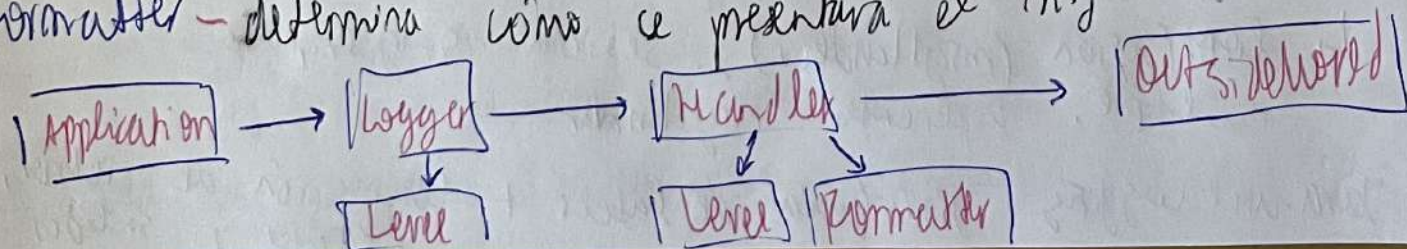
Arquitectura visible

Logger - objeto al que le pedimos que emita un msg de log

Handler - encargado de enviar el msg a donde comprenda

Level - indica la importancia de un msg y es lo que mira un logger y un handler para ver si le interesa

Formatter - determina cómo se presentará el msg

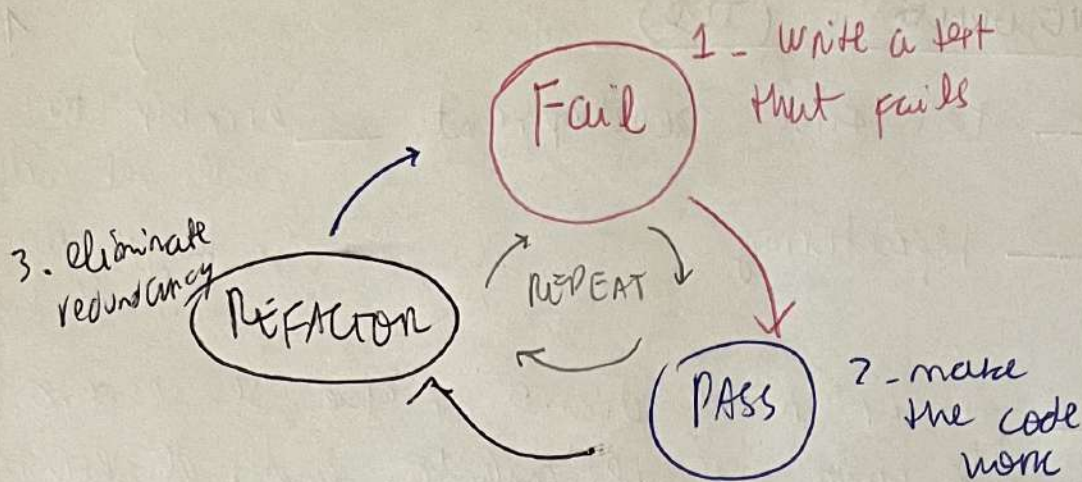


TEST DRIVEN DEVELOPMENT (TDD) extreme (programming) 10

- Combina — Test First Development — escribir test antes del código que haga pasar el test
 - refactoring
- objetivo — pensar en el diseño y qué se espera de cada requerimiento antes de escribir código
 - escribir código limpio que funcione (como técnica de programación)
- Granularidad del test
 - Test de unidad
 - reducir el tiempo/proceso entre requerimiento a código
 - aislar cada unidad de un programa y mostrar que funciona correctamente
 - escritos desde la perspectiva del programador
 - Test de aceptación
 - por cada funcionalidad esperada
 - escritos desde la perspectiva del cliente

Filosofía de TDD

- escribir tests primero, antes que el código
- escribir tests funcionales para capturar use cases que se validan automáticamente
- escribir tests de unidad (enfoque en pequeñas partes y aislar errores)
- no agregar funcionalidad hasta que ~~los~~ tests pasen
- una vez escrito el test, se codifica para que pase
- una vez que pasan los tests, se refactoriza para un código de calidad



métodos ágiles

- Se desarrolla en iteraciones
- cada iteración extiende código existente con nueva funcionalidad
- poca documentación
- testing automático e integración continua

JUNIT : es un framework

XUNIT Patterns : estrategias para escribir tests de unidad documentados como patrones